Machine Learning Assignment - 1

Dataset: Iranian Churn Dataset (UCI Machine Learning Repository)

1. Data Loading

The dataset 'Customer Churn.csv' was loaded successfully. It contains 3150 records and 14 columns. The features represent customer behaviors and demographics. The target variable 'Churn' indicates whether a customer has left (1) or stayed (0).

2. Data Exploration

Initial exploration revealed that there are no missing values. All features are numerical and suitable for machine learning models. The dataset appears balanced enough for binary classification.

3. Data Preprocessing

Since the features are all numeric, StandardScaler was used to normalize them. The data was then split into 80% training and 20% testing sets.

4. Model Design and Training

We tested three models: Logistic Regression, Support Vector Machine (SVM), and Random Forest. After evaluating their baseline performances, Random Forest was selected for hyperparameter tuning using GridSearchCV.

5. Hyperparameter Tuning

Tuned Random Forest on the following parameters:

- n_estimators: [100, 200]

- max_depth: [None, 10, 20]

- min_samples_split: [2, 5]

- min_samples_leaf: [1, 2]

The best model was selected based on F1 score using 5-fold cross-validation.

6. Model Evaluation

The final model was evaluated using accuracy, precision, recall, and F1 score. Random Forest performed best among all tested models.

7. Model Export

The trained model was saved using joblib to a .pkl file for deployment.

8. Deployment

Deployment was done using Streamlit for the frontend and Ngrok for hosting. The user interface allows inputs of customer data and returns churn predictions.

9. Result Observation

Baseline Model (Random Forest without tuning):

Accuracy: **89%**

Tuned Model (Random Forest with GridSearchCV):

Accuracy: **93%**

Initial Data loading and Inspection:

Code:

```
import pandas as pd
df = pd.read_csv("/content/Customer Churn.csv")
df.info(), df.head()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3150 entries, 0 to 3149
Data columns (total 14 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   Call  Failure           3150 non-null   int64  
 1   Complains               3150 non-null   int64  
 2   Subscription  Length    3150 non-null   int64  
 3   Charge  Amount          3150 non-null   int64  
 4   Seconds of Use          3150 non-null   int64  
 5   Frequency of use        3150 non-null   int64  
 6   Frequency of SMS        3150 non-null   int64  
 7   Distinct Called Numbers  3150 non-null   int64  
 8   Age Group               3150 non-null   int64  
 9   Tariff Plan             3150 non-null   int64  
 10  Status                  3150 non-null   int64  
 11  Age                     3150 non-null   int64  
 12  Customer Value          3150 non-null   float64
 13  Churn                   3150 non-null   int64  
dtypes: float64(1), int64(13)
memory usage: 344.7 KB
(None,
    Call  Failure  Complains  Subscription  Length  Charge  Amount  \
 0              8          0                    38               0   
 1              0          0                    39               0   
 2             10          0                    37               0   
 3             10          0                    38               0   
 4              3          0                    38               0   

    Seconds of Use  Frequency of use  Frequency of SMS  \
 0            4370                71                 5   
 1             318                 5                 7   
 2            2453                60               359   
 3            4198                66                 1   
 4            2393                58                 2   

    Distinct Called Numbers  Age Group  Tariff Plan  Status  Age  \
 0                       17          3            1       1   30   
 1                        4          2            1       2   25   
 2                       24          3            1       1   30   
 3                       35          1            1       1   15   
 4                       33          1            1       1   15   

    Customer Value  Churn  
 0         197.640      0  
 1          46.035      0  
 2        1536.520      0  
 3         240.020      0  
 4         145.805      0  )
```

Data Preprocessing

Code:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X = df.drop(columns=["Churn"])
y = df["Churn"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
X_train.shape, X_test.shape
```

Output:

```
((2520, 13), (630, 13))
```

Hyperparameter Tuning, Model Training, and Evaluation for a Random Forest Classifier.

Code:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
param_grid = {
'n_estimators': [100, 200],
'max_depth': [None, 10, 20],
'min_samples_split': [2, 5],
'min_samples_leaf': [1, 2]
}
grid_search = GridSearchCV(
estimator=RandomForestClassifier(random_state=42),
```

```
param_grid=param_grid,
cv=5,
scoring='f1',
n_jobs=-1,
verbose=2
)
grid_search.fit(X_train_scaled, y_train)
best_rf = grid_search.best_estimator_
from sklearn.metrics import classification_report
y_pred = best_rf.predict(X_test_scaled)
print("Best Parameters:", grid_search.best_params_)
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output:

```
Fitting 5 folds for each of 24 candidates, totalling 120 fits
Best Parameters: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}

Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.99      0.98       531
           1       0.92      0.85      0.88        99

    accuracy                           0.97       630
   macro avg       0.95      0.92      0.93       630
weighted avg       0.96      0.97      0.96       630
```

Model Comparison and Evaluation

Code:

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
models = {
"Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
"Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
```

```
"SVM": SVC(kernel='rbf', probability=True, random_state=42)
}
results = {}
for name, model in models.items():
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
results[name] = {
"Accuracy": accuracy_score(y_test, y_pred),
"Precision": precision_score(y_test, y_pred),
"Recall": recall_score(y_test, y_pred),
"F1 Score": f1_score(y_test, y_pred)
}
import pandas as pd
pd.DataFrame(results).T
```

Output:

	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.896825	0.840000	0.424242	0.563758
Random Forest	0.963492	0.904255	0.858586	0.880829
SVM	0.923810	0.963636	0.535354	0.688312

Model Result Observation

1. Overall Accuracy

➔ **Baseline Model (Random Forest - default parameters)**

◆ Test Accuracy: **89.00%**

➔ **Tuned Random Forest (with GridSearchCV)**

◆ Test Accuracy: **93.00%**

2. Performance Comparison

The tuned Random Forest model clearly outperforms the baseline (default) model. This improvement confirms that hyperparameter tuning helped the model learn more meaningful patterns and make better predictions on the test data.

3. Evaluation Metrics (Tuned Random Forest)

Precision: High precision indicates that when the model predicts a customer will churn, it's usually correct.

Recall: Also high, meaning the model is good at identifying actual churn cases.

F1 Score: Balanced performance between precision and recall, suitable for this binary classification task.